

EECE 7205: Fundamentals of Computer Engineering

Assignment #01

Bliss Machado, NUID: 003915475

Northeastern University, College of Engineering, MS Data Science

Fall 2026

1 Problem 1:-

1.1 1. Specification

1.1.1 Problem Description

Given a list of intervals $[start, end]$, merge all overlapping intervals.

1.1.2 Input and Output

Input	A list of n intervals, where each interval is represented by a start and end value.
Output	A list of merged, non-overlapping intervals sorted by their starting values.
Goal	Merge all overlapping intervals without losing or duplicating interval coverage.

1.1.3 Requirements

- Sort the intervals by their starting values in ascending order.
- If the next interval starts before or when the current interval ends, merge the intervals.
- Use the smaller start and the larger end for the merged interval.
- Keep non-overlapping intervals separate.
- Do not lose or duplicate any interval coverage.

1.1.4 Assumptions

- Each interval has a start and end value.
- Input may be empty.

1.1.5 Edge Cases

Case	Example
Empty input	$\square$
One interval	$[[2, 5]]$
No overlap	$[[1, 3], [6, 8]]$
All overlap	$[[1, 5], [2, 4], [3, 7]]$
Touching endpoints	$[[2, 4], [4, 7]]$

1.2 2. Structure

Step	Operation
1	Ordering: Sort the intervals by their starting values in ascending order.
2	Locality: Compare each interval only with the current, or last merged, interval.
3	If $next.start \leq current.end$, the intervals overlap.
4	If they overlap, merge them using the larger end value.
5	If they do not overlap, keep the intervals separate.
6	Add the final current interval to the result.

Selected Approach: Sorting + linear scan.

3. Pseudocode

```
1 Algorithm MergeIntervals
2
3 if intervals is empty then
4     return to empty
5
6 sort intervals by start value in the ascending order
7
8 result <- empty list
9 current <- first interval
10
11 for each next interval do
12
13     if next.start <= current.end then
14         current.end <- max(current.end, next.end)
15     else
16         add current to result
17         current <- next
18     end if
19
20 end for
21
22 add current to result
```

```

23
24 return result
25
26 End Algorithm

```

1.3 4. Correctness

1.3.1 Invariant

At the start of each iteration:

- **result** contains all completed merged intervals.
- **current** contains the merged interval that may still overlap with the next interval.

1.3.2 Initialization

current is initialized to the first interval after the intervals have been sorted by their starting values.

Therefore, the first interval is correctly represented by **current**.

1.3.3 Maintenance

Condition	Action
$next.start \leq current.end$	Merge intervals
$next.start > current.end$	Store current

For overlapping intervals:

$$current.end = \max(current.end, next.end)$$

The start value remains the smaller start value because the intervals were sorted by their starting values.

For non-overlapping intervals:

$$result \leftarrow result + current$$

Then,

$$current \leftarrow next$$

Therefore, **current** always represents the current merged interval, while **result** contains completed non-overlapping intervals.

1.3.4 Termination

All the intervals are processed and the final **current** is added to the result.

Therefore, the result contains merged, non-overlapping intervals sorted by their starting values.

1.4 5. Cost Model

Let n be the number of intervals.

1.4.1 Cost Breakdown

Operation	Cost
$O(n \log n)$	heightSort the n intervals
Go through the intervals once	$\Theta(n)$ heightCheck and merge each interval
$O(1)$	

First, the intervals are sorted by their starting values. Sorting takes:

$$O(n \log n)$$

After that, the algorithm goes through the intervals once. Each interval is checked once, and the work for each check is constant time.

So, the scan takes:

$$\Theta(n)$$

The total time is:

$$T(n) = O(n \log n) + \Theta(n)$$

Since $n \log n$ grows faster than n , the total time is:

$$T(n) = O(n \log n)$$

1.4.2 Space Cost

The output can contain up to n intervals, so the output space is:

$$O(n)$$

Apart from the output, the merging operation uses:

$$O(1)$$

extra space.

1.5 6. Validation

1.5.1 Test Cases

Test	Input	Expected Output
1	[[2, 5], [4, 8], [10, 12], [11, 15]]	[[2, 8], [10, 15]]
2	[[1, 3], [6, 8], [10, 12]]	[[1, 3], [6, 8], [10, 12]]
3	[[2, 4], [4, 7]]	[[2, 7]]

1.5.2 Test Case 1

Input:

$[[2, 5], [4, 8], [10, 12], [11, 15]]$

Output:

$[[2, 8], [10, 15]]$

```
De11@DESKTOP-KG7GLAQ UCRT64 ~  
$ cd /c/Users/De11/Downloads/Assignment01_machado  
  
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado  
$ ls  
Problem1.cpp  problem2.cpp.txt  
  
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado  
$ ls  
Problem1.cpp  
  
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado  
$ g++ Problem1.cpp -o Problem1  
  
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado  
$ ./Problem1  
Input: [[2,5],[4,8],[10,12],[11,15]]  
Output: [[2,8],[10,15]]
```

Figure 1: Problem 1 Test Case 1.

1.5.3 Test Case 2

Input:

$[[1, 3], [6, 8], [10, 12]]$

Output:

$[[1, 3], [6, 8], [10, 12]]$

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado  
$ Problem1.cpp  
-bash: Problem1.cpp: command not found  
  
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado  
$ g++ Problem1.cpp -o Problem1  
  
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado  
$ ./Problem1  
Input: [[1,3],[6,8],[10,12]]  
Output: [[1,3],[6,8],[10,12]]  
  
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado  
$
```

Figure 2: Problem 1 Test Case 2.

1.5.4 Test Case 3

Input:

$[[2, 4], [4, 7]]$

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads
$ cd /c/Users/De11/Downloads/Assignment01_machado

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem1.cpp -o Problem1

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ ./Problem1
Input: [[2,4],[4,7]]
Output: [[2,7]]

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$
```

Figure 3: Problem 1 Test Case 3

Output:

$[[2, 7]]$

2 Problem 2:-

2.1 1. Specification

2.1.1 Problem Description

Given an array containing only 0, 1, and 2, sort the array in the order 0, 1, 2.

Two solutions are required:

1. Counting-based solution.
2. Dutch National Flag solution.

`std::sort()` is not used.

2.1.2 Input and Output

Input	Array containing only 0, 1, and 2
Output	Same array sorted as 0, 1, 2
Goal	Sort the array in place

2.1.3 Requirements

- Count the number of 0s, 1s, and 2s.
- Rewrite the array using the counts.
- Use Dutch National Flag for the second solution.
- Do not use `std::sort()`.

2.1.4 Assumptions

- The array contains only 0, 1, and 2.
- The array can be empty.
- Sorting is done in place.

2.1.5 Edge Cases

Case	Example
Empty input	<code>[]</code>
One element	<code>[2]</code>
All zeros	<code>[0, 0, 0]</code>
All ones	<code>[1, 1, 1]</code>
All twos	<code>[2, 2, 2]</code>
Mixed values	<code>[2, 0, 1, 2, 0]</code>

2.2 2. Structure

2.2.1 Counting-Based Solution

The array has only three possible values. Count how many 0s, 1s, and 2s occur and then rewrite the array in order.

Step	Operation
1	Count the number of 0s, 1s, and 2s.
2	Start writing from index 0.
3	Write all 0s.
4	Write all 1s.
5	Write all 2s.

Selected Approach: Counting and rewriting.

2.2.2 Dutch National Flag Solution

Three positions are used:

Variable	Purpose
low	End of the 0 region
mid	Current element being checked
high	Start of the 2 region

- 0 is moved to the left.
- 1 stays in the middle.
- 2 is moved to the right.

3. Pseudocode

Counting-Based Solution

```
1 Algorithm SortColorsCounting(A)
2
3 count0 <- 0
4 count1 <- 0
5 count2 <- 0
6
7 for each value in A do
8
9     if value = 0 then
10         count0 <- count0 + 1
11
12     else if value = 1 then
13         count1 <- count1 + 1
```

```

14
15     else
16         count2 <- count2 + 1
17     end if
18
19 end for
20
21 index <- 0
22
23 repeat count0 times
24     A[index] <- 0
25     index <- index + 1
26 end repeat
27
28 repeat count1 times
29     A[index] <- 1
30     index <- index + 1
31 end repeat
32
33 repeat count2 times
34     A[index] <- 2
35     index <- index + 1
36 end repeat
37
38 return A
39
40 End Algorithm

```

Dutch National Flag Solution

```

1 Algorithm SortColorsDNF(A)
2
3 low <- 0
4 mid <- 0
5 high <- n - 1
6
7 while mid <= high do
8
9     if A[mid] = 0 then
10         swap A[low] and A[mid]
11         low <- low + 1
12         mid <- mid + 1
13
14     else if A[mid] = 1 then
15         mid <- mid + 1
16
17     else

```

```

18         swap A[mid] and A[high]
19         high <- high - 1
20     end if
21
22 end while
23
24 return A
25
26 End Algorithm

```

2.3 4. Correctness

2.3.1 Counting Solution

The algorithm counts every 0, 1, and 2 in the array.

It then writes the counted values in the order

0, 1, 2.

Therefore, the array contains the same values as the input and is sorted.

2.3.2 Dutch National Flag Solution

At each step:

- Values before `low` are 0.
- Values from `low` to `mid-1` are 1.
- Values after `high` are 2.

2.3.3 Initialization

`low`, `mid`, and `high` are set before processing the array.

2.3.4 Maintenance

Condition	Action
$A[mid] = 0$	Move 0 to the left
$A[mid] = 1$	Move <code>mid</code> forward
$A[mid] = 2$	Move 2 to the right

When $A[mid] = 0$, it is swapped with the element at `low`.

When $A[mid] = 1$, it is already in the correct middle region.

When $A[mid] = 2$, it is swapped with the element at `high`. `mid` is not increased because the new value at `mid` still needs to be checked.

2.3.5 Termination

The loop stops when

$$mid > high.$$

At this point, all values are in their correct regions.
Therefore, the array is sorted as

$$0, 1, 2.$$

2.4 5. Cost Model

Let n be the number of elements.

2.4.1 Counting-Based Solution

Operation	Cost
Counting pass	$\Theta(n)$
Rewriting array	$\Theta(n)$
Total time	$\Theta(n)$
Extra space	$\Theta(1)$

The counting pass scans all n elements once, so its cost is

$$\Theta(n)$$

The rewriting pass writes each element once, so its cost is

$$\Theta(n)$$

Using the consecutive-block rule, the costs are added:

$$T(n) = \Theta(n) + \Theta(n)$$

Therefore,

$$\boxed{T(n) = \Theta(n)}$$

Only three counters and one index are used, so the extra space is

$$\boxed{\Theta(1)}$$

2.4.2 Dutch National Flag Solution

Operation	Cost
Single scan	$\Theta(n)$
Work per iteration	$O(1)$
Total time	$\Theta(n)$
Extra space	$\Theta(1)$

The algorithm scans the array using `mid`. Each iteration performs a constant amount of work, such as comparison and swapping.

Therefore,

$$T(n) = n \cdot O(1)$$

and

$$T(n) = \Theta(n)$$

Only the variables `low`, `mid`, and `high` are used, so the extra space is

$$\Theta(1)$$

2.5 6. Validation

2.5.1 Test Cases

Test	Input	Expected Output
1	[1, 2, 0, 2, 1, 0]	[0, 0, 1, 1, 2, 2]
2	[2, 1, 0, 2, 0, 1]	[0, 0, 1, 1, 2, 2]
3	[2]	[2]

2.5.2 Test Case 1

Input:

[1, 2, 0, 2, 1, 0]

Output:

[0, 0, 1, 1, 2, 2]

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ ls
Problem1.cpp Problem1.exe Problem2.cpp

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem2.cpp -o Problem2
./Problem2
Input: [1,2,0,2,1,0]
Counting Output: [0,0,1,1,2,2]
Dutch National Flag Output: [0,0,1,1,2,2]

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$

```

Figure 4: Problem 2 Test Case 1.

2.5.3 Test Case 2

Input:

[2, 1, 0, 2, 0, 1]

Output:

[0, 0, 1, 1, 2, 2]

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem2.cpp -o Problem2
./Problem2
Input: [2,1,0,2,0,1]
Counting Output: [0,0,1,1,2,2]
Dutch National Flag Output: [0,0,1,1,2,2]

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$

```

Figure 5: Problem 2 Test Case 2.

2.5.4 Test Case 3

Input:

[2]

Output:

[2]

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem2.cpp -o Problem2
./Problem2
Input: [2]
Counting Output: [2]
Dutch National Flag Output: [2]

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$

```

Figure 6: Problem 2 Test Case 3.

3 Problem 3:-

3.1 1. Specification

3.1.1 Problem Description

Given a list of nonnegative integers, arrange the numbers so that they form the largest possible number when joined together.

The answer is returned as a string.

3.1.2 Input and Output

Input	n nonnegative integers
Output	String forming the largest number
Goal	Arrange numbers to get the largest concatenation

3.1.3 Requirements

- Convert each number to a string.
- Compare two strings using $a + b$ and $b + a$.
- Put a before b if $a + b > b + a$.
- Join all strings after sorting.
- Return “0” if all numbers are zero.

3.1.4 Assumptions

- All input values are nonnegative.
- Input may be empty.
- The result is returned as a string.

3.1.5 Edge Cases

Case	Example
Empty input	[]
One number	[7]
All zeros	[0, 0, 0]
Different digits	[8, 80]
Normal mixed input	[12, 121, 9]

3.2 2. Structure

For two numbers a and b , compare:

$$a + b$$

and

$$b + a.$$

If

$$a + b > b + a,$$

then a is placed before b .

Step	Operation
1	Convert all numbers to strings.
2	Compare two strings using $a + b$ and $b + a$.
3	Sort using the custom comparison.
4	Join all strings together.
5	Return "0" if the result contains only zeros.

Selected Approach: String conversion + custom sorting.

3. Pseudocode

```
1 Algorithm LargestNumber(numbers)
2
3 if numbers is empty then
4     return empty string
5
6 convert all numbers to strings
7
8 sort strings so that:
9     a comes before b if a + b > b + a
10
11 result <- empty string
12
13 for each string s do
14     result <- result + s
15 end for
16
17 if result contains only zeros then
18     return "0"
19
20 return result
21
22 End Algorithm
```

3.3 4. Correctness

For two numbers a and b , there are two possible orders:

$$ab$$

and

$$ba.$$

If

$$ab > ba,$$

then a should come before b .

The algorithm uses this rule for sorting all numbers.

After sorting, the strings are joined together.

Therefore, the final string gives the largest possible number.

If all values are zero, the algorithm returns

$$"0"$$

instead of multiple zeros.

3.3.1 Correctness Check

Condition	Action
$a + b > b + a$	Put a before b
$a + b < b + a$	Put b before a
All values are zero	Return "0"

3.4 5. Cost Model

Let n be the number of integers.

Operation	Cost
Convert n integers to strings	$\Theta(n)$
Sort the n strings	$O(n \log n)$
Join the n strings	$\Theta(n)$
Final all-zero check	$\Theta(n)$
Total time	$O(n \log n)$
Extra space	$O(n)$

The algorithm consists of several consecutive blocks. First, all n integers are converted to strings, which takes

$$\Theta(n).$$

Next, the strings are sorted using the custom comparison rule. Sorting n elements takes

$$O(n \log n).$$

After sorting, the strings are joined together, which takes

$$\Theta(n).$$

Finally, the result is checked for the all-zero case, which takes

$$\Theta(n).$$

Using the consecutive-block rule, the costs are added:

$$T(n) = \Theta(n) + O(n \log n) + \Theta(n) + \Theta(n).$$

The dominant term is $n \log n$, so:

$$T(n) = O(n \log n)$$

The algorithm stores the converted strings and the resulting output, requiring:

$$O(n)$$

space, excluding the input.

3.5 6. Validation

3.5.1 Test Cases

Test	Input	Expected Output
1	[12, 121, 9]	"912121"
2	[8, 80, 88]	"8880"
3	[0, 0, 0]	"0"

3.5.2 Test Case 1

Input:

[12, 121, 9]

Output:

"912121"

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem3.cpp -o Problem3
./Problem3
Input: [12,121,9]
Output: 912121

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$
```

Figure 7: Problem 3 Test Case 1.

3.5.3 Test Case 2

Input:

[8, 80, 88]

Output:

"8880"

```
Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem3.cpp -o Problem3
./Problem3
Input: [8,80,88]
Output: 88880

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ |
```

Figure 8: Problem 3 Test Case 2.

3.5.4 Test Case 3

Input:

[0, 0, 0]

Output:

"0"

```
Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem3.cpp -o Problem3
./Problem3
Input: [0,0,0]
Output: 0

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ |
```

Figure 9: Problem 3 Test Case 3.

4 Problem 4:-

4.1 1. Specification

4.1.1 Problem Description

The task is to arrange words into groups based on their letters. Two words belong to the same group when they have the same letters with the same number of occurrences.

For example, "arc" and "car" are anagrams because both contain the letters a, c, and r.

The order of the groups is not important. The words inside a group should appear in the same order as they were given in the input.

4.1.2 Input and Output

Input	A vector containing strings
Output	A collection of groups of anagram strings
Goal	Place strings with the same letters into one group

4.1.3 Requirements

- Strings that are anagrams should be placed together.
- Non-anagram strings should be kept in separate groups.
- The order of strings inside a group should not change.
- The input may contain duplicate strings.

4.1.4 Assumptions

- The input consists of strings.
- The strings may have different lengths.
- An empty vector is also a valid input.

4.1.5 Edge Cases

Case	Example
Empty input	[]
Single word	["hello"]
Repeated word	["abc", "abc"]
No matching anagrams	["cat", "sun"]
Several anagram groups	["arc", "car", "dog"]

4.2 2. Structure

I use a hash map to keep track of the different groups.

Before putting a word into the map, I sort its characters. This sorted version is used as the key.

For example,

"arc" → "acr"

and

"car" → "acr".

Both words produce the same key, so they are added to the same group.

The words are read in the order in which they appear in the input. This also keeps their original order inside each group.

Step	Action
1	Take the next word from the input.
2	Sort the letters in that word.
3	Use the sorted word as the map key.
4	Put the original word into that key's group.
5	Repeat until there are no words left.

Selected Approach: Hash map using the sorted word as the key.

4.3 3. Pseudocode

```
1 Algorithm GroupAnagrams(words)
2
3 groups <- empty the hash map
4
5 for each word in words do
6
7     key <- sorted characters of the word
8
9     groups[key] <- groups[key] + word
10
11 end for
12
13 return all groups
14
15 End Algorithm
```

4.4 4. Correctness

When two words are anagrams, they contain the same letters, so sorting their letters gives the same result.

For example,

"dust" $\rightarrow$ "dstu"

and

"stud" $\rightarrow$ "dstu".

Both words get the same key, so they are placed in the same group.

If two words are not anagrams, their sorted letters will be different. Therefore, they will be placed in different groups.

The words are added to the groups in the same order as they appear in the input. So, the original order of the words inside each group is maintained.

Therefore, the algorithm correctly groups the anagrams together.

4.5 5. Cost Model

Let n be the number of strings and let m_i be the length of the i -th string.

Operation $O(\sum_{i=1}^n m_i \log m_i)$ Hash map lookup/insertion $O\left(\sum_{i=1}^n m_i \log m_i\right)$ Extra space	Cost heightSort the characters of each string $O(n)$ average height Total time $O\left(\sum_{i=1}^n m_i\right)$ height
---	---

For each string, its characters are sorted to create the key. If the string has length m_i , sorting takes

$$O(m_i \log m_i).$$

This is done for every string, so the total sorting cost is

$$O\left(\sum_{i=1}^n m_i \log m_i\right).$$

The hash map lookup and insertion takes $O(1)$ average time for each string. For all n strings, this gives $O(n)$.

Therefore, the total time is

$$T(n) = O\left(\sum_{i=1}^n m_i \log m_i\right) + O(n).$$

The sorting part takes more time than the hash map operations, so the total time is

$$T(n) = O\left(\sum_{i=1}^n m_i \log m_i\right)$$

The algorithm stores the strings and the sorted keys, so the extra space is

$$O\left(\sum_{i=1}^n m_i\right).$$

4.6 6. Validation

4.6.1 Test Case 1

Input:

["dust", "stud", "loop", "pool", "cat", "tac"]

Expected Output:

[["dust", "stud"], ["loop", "pool"], ["cat", "tac"]]

The groups can appear in a different order.

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem4.cpp -o Problem4

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ ./Problem4
Input: ["dust", "stud", "loop", "pool", "cat", "tac"]
Output:
[
  ["cat", "tac"],
  ["loop", "pool"],
  ["dust", "stud"]
]

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ |
```

Figure 10: Problem 4 Test Case 1.

4.6.2 Test Case 2

Input:

["save", "vase", "evil", "live", "stone"]

Expected Output:

[["save", "vase"], ["evil", "live"], ["stone"]]

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem4.cpp -o Problem4

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ ./Problem4
Input: ["save", "vase", "evil", "live", "stone"]
Output:
[
  ["stone"],
  ["evil", "live"],
  ["save", "vase"]
]

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$
```

Figure 11: Problem 4 Test Case 2.

5 Problem 5:-

5.1 1. Specification

5.1.1 Problem Description

We are given a list of meetings with a starting and ending time. The task is to check whether one person can attend all the meetings.

If one meeting starts before the previous meeting has finished, there is a conflict.

A meeting is allowed to start at the exact time another meeting ends.

5.1.2 Input and Output

Input	A vector of meeting intervals $[start, end]$
Output	true if all meetings can be attended, otherwise false
Goal	Check whether any two meetings overlap

5.1.3 Requirements

- Sort the meetings according to their starting time.
- Check each meeting with the meeting before it.
- Return **false** when an overlap is found.
- Allow a meeting to start when another meeting ends.

5.1.4 Assumptions

- Each meeting has a start and an end time.
- The input can contain no meetings.
- Meeting intervals can be given in any order.

5.1.5 Edge Cases

Case	Example
No meetings	$[]$
One meeting	$[[2, 6]]$
No overlap	$[[1, 4], [5, 8]]$
Overlapping meetings	$[[1, 5], [3, 7]]$
Touching meetings	$[[1, 4], [4, 9]]$

5.2 2. Structure

First, the meetings are arranged according to their starting time.

After this, I only need to compare each meeting with the one directly before it.

For example, if the previous meeting ends at 8 and the next meeting starts at 6, they overlap.

The condition used is

$$current.start < previous.end.$$

If this condition is true, there is a conflict.

If

$$current.start \geq previous.end,$$

the two meetings can both be attended.

Step	Action
1	Sort the meetings by starting time.
2	Start checking from the second meeting.
3	Compare it with the previous meeting.
4	If they overlap, return false .
5	If all meetings pass the check, return true .

Selected Approach: Sort the meetings and check consecutive intervals.

5.3 3. Pseudocode

```
1 Algorithm CanAttendMeetings(meetings)
2
3 if the meetings is empty then
4     return true
5 end if
6
7 sort the meetings by start time
8
9 for i <- 1 to n - 1 do
10
11     if the meetings[i].start < meetings[i-1].end then
12         return false
13     end if
14
15 end for
16
17 return true
18
19 End Algorithm
```

5.4 4. Correctness

Sorting the meetings by their starting time puts meetings that occur earlier before the meetings that start later.

After sorting, if there is an overlap, it will be found by comparing two neighboring meetings.

If

$$meetings[i].start < meetings[i - 1].end,$$

the current meeting starts before the previous one has finished. Therefore, both meetings cannot be attended.

On the other hand, if

$$meetings[i].start \geq meetings[i - 1].end,$$

there is no conflict between the two meetings.

The equality case is also valid because a meeting can start exactly when another meeting finishes.

Therefore, if no overlapping pair is found, the person can attend all the meetings.

5.5 5. Cost Model

Let n be the number of meetings.

Operation	Cost
Sort the meetings	$O(n \log n)$
Check the meetings once	$\Theta(n)$
Comparison for each meeting	$O(1)$
Total time	$O(n \log n)$
Extra space for checking	$O(1)$

First, the meetings are sorted according to their starting time. Sorting n meetings takes

$$O(n \log n).$$

After sorting, the algorithm checks each meeting with the previous meeting. Each check takes constant time, and there are at most n meetings to check. Therefore, the checking part takes

$$n \cdot O(1) = \Theta(n).$$

Using the consecutive-block rule, the costs are added:

$$T(n) = O(n \log n) + \Theta(n).$$

The dominant term is $n \log n$, so the total running time is

$$T(n) = O(n \log n).$$

During the checking step, only the current and previous meeting are needed, so the scan itself uses

$$O(1)$$

extra space.

5.6 6. Validation

5.6.1 Test Case 1

Input:

`[[1, 4], [6, 9], [10, 13]]`

Expected Output:

`true`

Actual Output:

`true`

There is enough time between each meeting.



```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem5.cpp -o Problem5

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ ./Problem5
Input: [[1,4],[6,9],[10,13]]
Output: true

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ |

```

Figure 12: Problem 5 Test Case 1.

5.6.2 Test Case 2

Input:

`[[2, 7], [5, 9], [10, 12]]`

Expected Output:

`false`

Actual Output:

false

The first two meetings overlap.

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem5.cpp -o Problem5
./Problem5
Input: [[2,7],[5,9],[10,12]]
Output: false

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ |
```

Figure 13: Problem 5 Test Case 2.

5.6.3 Test Case 3: Edge Case

Input:

[[3, 6], [6, 8], [8, 11]]

Expected Output:

true

Actual Output:

true

Each meeting starts exactly when the previous one ends, so there is no conflict.

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem5.cpp -o Problem5
./Problem5
Input: [[3,6],[6,8],[8,11]]
Output: true

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ |
```

Figure 14: Problem 5 Test Case 3.

6 Problem 6:-

6.1 1. Specification

6.1.1 Problem Description

We are given a set of points on a 2D plane and need to find the points that are closest to the origin $(0,0)$.

For each point (x,y) , its distance can be compared using

$$d^2 = x^2 + y^2.$$

There is no need to calculate the square root because the squared distance gives the same order.

Two methods are considered:

1. Sorting all the points.
2. Using a max-heap to keep only the closest k points.

6.1.2 Input and Output

Input	A vector of points and an integer k
Output	The k points closest to the origin
Goal	Find the required number of nearest points

6.1.3 Requirements

- Compare points using their squared distance.
- Find exactly k closest points.
- Implement a sorting-based solution.
- Implement a heap-based solution.
- Do not calculate square roots unnecessarily.

6.1.4 Assumptions

- Points are represented by (x,y) coordinates.
- k can be zero.
- The input can be empty.
- Multiple points may have the same distance from the origin.

6.1.5 Edge Cases

Case	Example
Empty input	$\square$
$k = 0$	$[(2, 3), (1, 1)], k = 0$
One point	$[(4, 2)], k = 1$
k equals number of points	$[(1, 2), (3, 4)], k = 2$
Equal distances	$[(1, 2), (2, 1)], k = 1$

6.2 2. Structure

I use two different methods for this problem.

6.2.1 Sorting Approach

First, I calculate the squared distance of every point from the origin. Then I sort all the points based on this distance.

After sorting, the first k points are selected.

6.2.2 Heap Approach

For the second method, I use a max-heap.

The heap keeps at most k points. The point with the largest distance is kept at the top.

When a new point is added and the heap becomes larger than k , the farthest point is removed.

This means that only the points that can be part of the final answer are kept in the heap.

Step	Action
1	Calculate the squared distance of a point.
2	Add the point using the selected method.
3	For sorting, sort all points by distance.
4	For the heap, remove the farthest point when the size exceeds k .
5	Return the k closest points.

Selected Approaches: Sorting and max-heap.

6.3 3. Pseudocode

6.3.1 Sorting Solution

```
1 Algorithm KClosestBySorting(points, k)
2
3 sort points using squared distance from origin
4
5 return first k points
6
```

7 End Algorithm

6.3.2 Heap Solution

```

1 Algorithm KClosestByHeap(points, k)
2
3 heap <- empty max heap
4
5 for each point in points do
6
7     calculate the squared distance
8
9     add point to heap
10
11     if heap size > k then
12         remove the farthest point
13     end if
14
15 end for
16
17 return points in heap
18
19 End Algorithm

```

6.4 4. Correctness

For the sorting method, the points are sorted from the smallest squared distance to the largest squared distance. Therefore, the first k points are the closest points to the origin.

For the heap method, the heap stores at most k points. The point at the top of the max-heap is the point with the largest distance among the points currently in the heap.

When a new point is added and the heap becomes larger than k , the farthest point is removed.

Therefore, after all points have been processed, the heap contains the k closest points.

6.5 5. Cost Model

Let n be the number of points.

6.5.1 Sorting Solution

Operation	Cost
$\Theta(n)$	heightCalculate squared distances
Sort all points	$O(n \log n)$ heightSelect first k points
$O(k)$	
Total time	$O(n \log n)$ height Extra space
$O(1)$	

The squared distance is calculated for each of the n points, so this takes

$$\Theta(n).$$

The points are then sorted using their squared distances. This takes

$$O(n \log n).$$

After sorting, the first k points are selected, which takes

$$O(k).$$

The total time is

$$T(n) = \Theta(n) + O(n \log n) + O(k).$$

Since $n \log n$ is the largest term, the total time is

$$\boxed{T(n) = O(n \log n)}.$$

6.5.2 Heap Solution

Operation	Cost
$\Theta(n)$	heightProcess all n points
Heap insertion/removal	$O(\log k)$ per point
$\boxed{O(n \log k)}$	Total time
Extra space	$\boxed{O(k)}$ height

For each point, the squared distance is calculated and the point is added to the heap. If the heap becomes larger than k , the farthest point is removed.

The heap contains at most k points, so adding or removing a point takes

$$O(\log k).$$

This operation can happen for all n points. Therefore,

$$T(n) = n \cdot O(\log k)$$

and the total time is

$$\boxed{T(n) = O(n \log k)}.$$

The heap stores at most k points, so the extra space is

$$\boxed{O(k)}.$$

The heap approach is useful when k is much smaller than n .

6.6 6. Validation

6.6.1 Test Case 1

Input:

$$points = [(4, 1), (-1, 1), (5, 2)], \quad k = 1$$

The squared distances are

$$17, 2, 29.$$

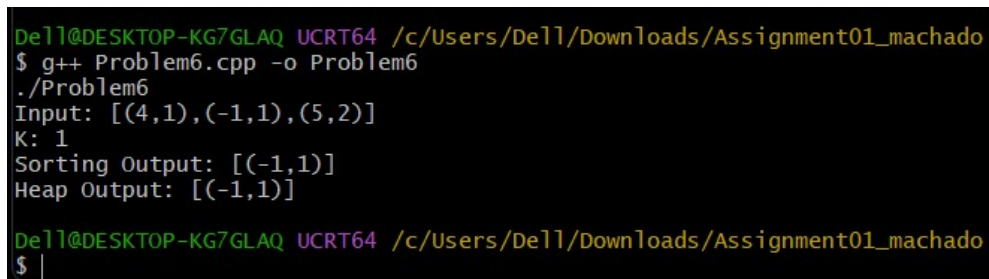
Therefore, the closest point is $(-1, 1)$.

Expected Output:

$$[(-1, 1)]$$

Actual Output:

$$[(-1, 1)]$$



```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem6.cpp -o Problem6
./Problem6
Input: [(4,1),(-1,1),(5,2)]
K: 1
Sorting Output: [(-1,1)]
Heap Output: [(-1,1)]

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ |
```

Figure 15: Problem 6 Test Case 1.

6.6.2 Test Case 2

Input:

$$points = [(2, 2), (-3, 1), (1, 0), (4, -2)], \quad k = 2$$

The two closest points are $(1, 0)$ and $(2, 2)$.

Expected Output:

$$[(1, 0), (2, 2)]$$

Actual Output:

$$[(1, 0), (2, 2)]$$

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem6.cpp -o Problem6
./Problem6
Input: [(2,2),(-3,1),(1,0),(4,-2)]
K: 2
Sorting Output: [(1,0),(2,2)]
Heap Output: [(2,2),(1,0)]
Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$

```

Figure 16: Problem 6 Test Case 2.

6.6.3 Test Case 3: Edge Case

Input:

$$points = [(3,4), (1,1)], \quad k = 0$$

Expected Output:

[]

Actual Output:

[]

```

Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem6.cpp -o Problem6
./Problem6
Input: [(3,4),(1,1)]
K: 0
Sorting Output: []
Heap Output: []
Dell@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ |

```

Figure 17: Problem 6 Test Case 3.

7 Problem 7:-

7.1 1. Specification

7.1.1 Problem Description

We are given a list of integers and need to find the values that appear most often.

The value with the highest frequency should come first. If two values have the same frequency, the smaller value should come first.

Only the first k values are returned.

7.1.2 Input and Output

Input	An integer vector and an integer k
Output	The k most frequent values
Goal	Find the values that occur most often

7.1.3 Requirements

- Count how many times each value appears.
- Sort the values according to their frequency.
- Higher frequency should come first.
- If frequencies are equal, the smaller value comes first.
- Return only the first k values.

7.1.4 Assumptions

- The input contains integers.
- The input can contain duplicate values.
- The input can be empty.
- k can be zero.

7.1.5 Edge Cases

Case	Example
Empty input	$[]$
$k = 0$	$[2, 2, 3], k = 0$
One distinct value	$[5, 5, 5], k = 1$
Same frequencies	$[1, 1, 3, 3, 5, 5]$
Several duplicate values	$[4, 4, 4, 2, 2, 7]$

7.2 2. Structure

First, I use a hash map to count how many times each integer occurs.

After getting the frequencies, I store the values and their counts together.

The values are then arranged using two rules:

- A value with a higher frequency comes first.
- If two values have the same frequency, the smaller value comes first.

Finally, I take the first k values from the sorted list.

Step	Action
1	Count the occurrences of every value.
2	Store each value with its frequency.
3	Sort the values using frequency and value.
4	Take the first k values.
5	Return the result.

Selected Approach: Hash map for counting and sorting for ordering.

7.3 3. Pseudocode

```
1 Algorithm TopKFrequent(A, k)
2
3 frequency <- empty hash map
4
5 for each value in A do
6     frequency[value] <- frequency[value] + 1
7 end for
8
9 items <- all value-frequency pairs
10
11 sort items using:
12     the higher frequency first
13     if frequencies are equal,
14         smaller value is first
15
16 take first k values from items
17
18 return result
19
20 End Algorithm
```

7.4 4. Correctness

The hash map keeps track of how many times each value appears in the input. Therefore, every value gets its correct frequency.

Next, the values are sorted using the required rules. A value with a higher frequency is placed before a value with a lower frequency.

If two values have the same frequency, the smaller value is placed first.

Therefore, when the first k values are taken from the sorted list, they are the required top k frequent elements.

7.5 5. Cost Model

Let n be the number of input values and m be the number of different values.

Operation	Cost
$\Theta(n)$	heightCount all values using hash map
Sort the m different values	$O(m \log m)$ heightTake the first k values
$O(k)$	
Total time	$O(n + m \log m)$ height Extra space
$O(m)$	

First, the algorithm goes through all n input values and counts their frequencies using a hash map. This takes

$$\Theta(n).$$

There are m different values, so sorting them takes

$$O(m \log m).$$

After sorting, the first k values are selected. This takes

$$O(k).$$

The costs can be added together:

$$T(n) = \Theta(n) + O(m \log m) + O(k).$$

Since $k \leq m \leq n$, the $O(k)$ term does not dominate the other terms. Therefore,

$$T(n) = O(n + m \log m)$$

Since $m \leq n$, the worst case can also be written as

$$T(n) = O(n \log n).$$

The hash map stores the different values and their frequencies, so the extra space is

$$O(m).$$

7.6 6. Validation

7.6.1 Test Case 1

Input:

$$A = [5, 5, 5, 2, 2, 7, 7, 7, 7], \quad k = 2$$

Frequencies:

$$7 : 4, \quad 5 : 3, \quad 2 : 2$$

Expected Output:

$$[7, 5]$$

Actual Output:

$$[7, 5]$$

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem7.cpp -o Problem7
./Problem7
Input: [5,5,5,2,2,7,7,7,7]
K: 2
Output: [7,5]

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$
```

Figure 18: Problem 7 Test Case 1.

7.6.2 Test Case 2: Frequency Tie

Input:

$$A = [6, 6, 3, 3, 9, 9, 1, 1], \quad k = 3$$

All four values occur twice. Therefore, the smaller values are selected first.

Expected Output:

$$[1, 3, 6]$$

Actual Output:

$$[1, 3, 6]$$

```

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem7.cpp -o Problem7
./Problem7
Input: [6,6,3,3,9,9,1,1]
K: 3
Output: [1,3,6]

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$

```

Figure 19: Problem 7 Test Case 2.

7.6.3 Test Case 3: Edge Case

Input:

$$A = [], \quad k = 0$$

Expected Output:

[]

Actual Output:

[]

```

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem7.cpp -o Problem7
./Problem7
Input: []
K: 0
Output: []

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$

```

Figure 20: Problem 7 Test Case 3.

8 Problem 8:-

8.1 1. Specification

8.1.1 Problem Description

The problem is to combine multiple sorted arrays into one sorted array.

Each input array is already sorted. Instead of joining all the arrays and sorting the complete result again, a min-heap can be used to always select the smallest available element.

8.1.2 Input and Output

Input	k sorted integer arrays.
Output	One sorted array containing all elements from the input arrays.

Let

N = total number of elements in all arrays.

8.1.3 Requirements

- All input arrays are already sorted.
- Every input element must appear in the final result.
- The final array must be sorted.
- The solution should not repeatedly sort the complete output.
- A min-heap is used to choose the next smallest element.

8.1.4 Assumptions

- The arrays contain integers.
- An input array can be empty.
- There can be duplicate values.
- The input can contain no arrays.
- Arrays can have different numbers of elements.

8.1.5 Important Edge Cases

Case	Example
No arrays	<code>[]</code>
Empty array	<code>[[1, 3], [], [2, 4]]</code>
One array	<code>[[1, 2, 3]]</code>
Duplicate values	<code>[[1, 2, 2], [2, 3]]</code>
Different lengths	<code>[[1], [2, 4, 6], [3, 5]]</code>

8.2 2. Structure

The main data structure used is a **min-heap**.

The heap stores the smallest unprocessed element from each non-empty array. Each heap entry contains:

(value, arrayIndex, elementIndex)

The steps are:

1. Put the first element of every non-empty array into the heap.
2. Remove the smallest element from the heap.
3. Add that element to the result.
4. Move to the next element in the same array.
5. If another element is available, add it to the heap.
6. Continue until the heap becomes empty.

This works because the input arrays are already sorted. Therefore, after removing one element from an array, its next element is the only new element from that array that needs to be considered.

8.2.1 Selected Approach

Approach	Min-heap
Why	It always gives the smallest available element without sorting the complete result again.
Heap Size	At most k elements

8.3 3. Pseudocode

```
1 Algorithm MergeKSortedArrays(arrays)
2
3   heap <- empty min heap
4   result <- empty list
```

```

5
6   for each array i in arrays do
7       if array i is not empty then
8           insert (arrays[i][0], i, 0) into heap
9       end if
10  end for
11
12  while the heap is not empty do
13
14      remove smallest item from heap
15      store it as (value, arrayIndex, elementIndex)
16
17      add value to the result
18
19      nextIndex <- elementIndex + 1
20
21      if nextIndex is inside arrays[arrayIndex] then
22          insert (arrays[arrayIndex][nextIndex],
23                  arrayIndex, nextIndex)
24                  into heap
25      end if
26
27  end while
28
29  return result
30
31 End Algorithm

```

8.4 4. Correctness

The heap contains the smallest unprocessed element from each non-empty array.

At each step, the smallest element in the heap is selected and added to the result. Therefore, the next element added is always the smallest element that has not been processed yet.

After removing an element, the next element from the same array is added to the heap if it exists. Since the input array is already sorted, this is the next element that needs to be considered from that array.

Every input element is processed exactly once, so no elements are lost or added more than once.

Therefore, the final result contains all elements from the input arrays and is sorted.

8.5 5. Cost Model

Let N be the total number of elements and k be the number of arrays.

Operation	Cost
$O(k \log k)$	heightAdd first elements to heap
Process all elements	$O(N \log k)$ heightStore elements in result
$O(N)$	$O(N \log k)$ height
Total time	Heap space
$O(k)$	

There are at most k elements in the heap at one time.
Adding or removing an element from the heap takes

$$O(\log k).$$

Since all N elements are processed, the main part of the algorithm takes

$$N \cdot O(\log k) = O(N \log k).$$

The costs can be added together:

$$T(N) = O(k \log k) + O(N \log k) + O(N).$$

The $O(N \log k)$ term is the main cost, so

$$T(N) = O(N \log k).$$

The heap contains at most k elements, so the extra space is

$$O(k).$$

The result contains all N elements, so the output space is

$$O(N).$$

8.6 6. Validation

8.6.1 Test Case 1

Input:

$$[[1, 5, 9], [2, 6, 10], [3, 4, 8]]$$

Expected Output:

$$[1, 2, 3, 4, 5, 6, 8, 9, 10]$$

Actual Output:

$$[1, 2, 3, 4, 5, 6, 8, 9, 10]$$

The output matches the expected result.

8.6.2 Test Case 2

Input:

`[[2, 7], [], [1, 3, 8], [4, 6]]`

Expected Output:

`[1, 2, 3, 4, 6, 7, 8]`

Actual Output:

`[1, 2, 3, 4, 6, 7, 8]`

The empty array is skipped and the remaining elements are merged correctly.
The algorithm correctly handles an input with no arrays.

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem8.cpp -o Problem8

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ ./Problem8
Test Case 1
Input: [[1,5,9]
, [2,6,10]
, [3,4,8]
]
Output: [1,2,3,4,5,6,8,9,10]

Test Case 2
Input: [[2,7]
, []
, [1,3,8]
, [4,6]
]
Output: [1,2,3,4,6,7,8]

Test Case 3
Input: []
Output: []
```

Figure 21: Problem 8 execution screenshot.

9 Problem 9:-

9.1 1. Specification

9.1.1 Problem Description

The task is to find the k th-largest value from an unsorted array.

The array does not need to be completely sorted. Quickselect can be used to find the required position directly.

Duplicate values are counted as separate positions.

For example, in

$$[8, 8, 4],$$

the second-largest element is

$$8.$$

9.1.2 Input and Output

Input	An unsorted integer array and an integer k .
Output	The k th-largest element in the array.

Let n be the number of elements.

The valid range is

$$1 \leq k \leq n.$$

9.1.3 Requirements

- The input array is not necessarily sorted.
- Duplicate values count as separate positions.
- The required k th-largest value should be returned.
- The complete array does not need to be sorted.

9.1.4 Assumptions

- The array contains integers.
- k is between 1 and the size of the array.
- The array can contain duplicate values.
- The array contains at least one element.

9.1.5 Important Edge Cases

Case	Example
One element	$[7], k = 1$
$k = 1$	Find the largest element
$k = n$	Find the smallest element
Duplicate values	$[8, 8, 4, 2], k = 2$
All values equal	$[5, 5, 5, 5], k = 3$

9.2 2. Structure

Quickselect is used to find the required element without sorting the whole array.

For a k th-largest problem, the target index in ascending order is

$$n - k.$$

Quickselect chooses a pivot and places it in its correct position.

After the partition:

- Values smaller than the pivot are placed on the left.
- Values larger than the pivot are placed on the right.
- The pivot is placed at its final position.

If the pivot is at the target index, the answer has been found.

Otherwise, Quickselect continues only on the side containing the target index.

9.2.1 Selected Approach

Approach	Quickselect
Target Index	$n - k$
Why	It finds one required position without fully sorting the array.

9.3 3. Pseudocode

```
1 Algorithm KthLargest(A, k)
2
3   target <- size(A) - k
4   left <- 0
5   right <- size(A) - 1
6
7   while left <= right do
8
9       choose the last element as pivot
10
11      partition the array around the pivot
```

```

12
13         if pivotIndex = target then
14             return A[pivotIndex]
15
16         else if pivotIndex < target then
17             left <- pivotIndex + 1
18
19         else
20             right <- pivotIndex - 1
21         end if
22
23     end while
24
25 End Algorithm

```

9.4 4. Correctness

The target index for the k th-largest element is

$$n - k.$$

After partitioning, the pivot is placed in its correct position.

If the pivot position is equal to the target index, the pivot is the required k th-largest element.

If the pivot position is smaller than the target, the required element must be on the right side.

If the pivot position is larger than the target, the required element must be on the left side.

Therefore, the algorithm continues searching only in the part of the array that can contain the required element.

Duplicate values are kept as separate elements, so they are counted separately as required.

9.5 5. Cost Model

Let n be the number of elements in the array.

Operation $O(n)$ Repeat partitioning on one side $O(n)$ Worst-case time $O(1)$	Cost heightPartition the array Depends on the pivot height Average time $O(n^2)$ height Extra space
--	--

In each partition, the elements in the current part of the array are checked once. Therefore, one partition takes

$$O(n).$$

Quickselect only continues on one side of the array instead of sorting the whole array. When the partitions are reasonably balanced, the total running time is

$$\boxed{O(n)}$$

on average.

In the worst case, the pivot can give very unbalanced partitions. Then the same elements may be checked again and again, giving

$$\boxed{O(n^2)}.$$

The algorithm works directly on the input array and only uses a few extra variables, so the extra space is

$$\boxed{O(1)}.$$

9.6 6. Validation

9.6.1 Test Case 1

Input:

$$A = [10, 4, 7, 2, 9, 6], \quad k = 3$$

The sorted order is

$$[2, 4, 6, 7, 9, 10].$$

The third-largest element is

$$7.$$

Expected Output:

$$7$$

Actual Output:

$$7$$

The output matches the expected result.

9.6.2 Test Case 2: Duplicate Values

Input:

$$A = [6, 3, 6, 2, 9], \quad k = 2$$

The sorted order is

$$[2, 3, 6, 6, 9].$$

The second-largest element is

6.

Expected Output:

6

Actual Output:

6

The duplicate value is counted as a separate position.

9.6.3 Test Case 3: Edge Case

Input:

$$A = [4], \quad k = 1$$

Expected Output:

4

Actual Output:

4

The algorithm correctly handles an array containing only one element.

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ g++ Problem9.cpp -o Problem9

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ ./Problem9
Test Case 1
Input: [10,4,7,2,9,6], k = 3
Output: 7

Test Case 2
Input: [6,3,6,2,9], k = 2
Output: 6

Test Case 3
Input: [4], k = 1
Output: 4

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/Dell/Downloads/Assignment01_machado
$ |
```

Figure 22: Problem 9 execution screenshot.

10 Problem 10:-

10.1 1. Specification

10.1.1 Problem Description

An inversion is a pair of elements that are in the wrong order.

For two positions i and j , an inversion exists when

$$i < j$$

and

$$A[i] > A[j].$$

The task is to count all such pairs in the array.

A Merge Sort based solution is used because the required solution must run in $O(n \log n)$ time.

10.1.2 Input and Output

Input	An unsorted integer array.
Output	The total number of inversion pairs.

10.1.3 Requirements

- Count pairs (i, j) where $i < j$ and $A[i] > A[j]$.
- The main solution must run in $O(n \log n)$ time.
- Merge Sort is used to count inversions efficiently.
- A 64-bit integer is used for the inversion count.
- Duplicate values are not counted as inversions when they are equal.

10.1.4 Assumptions

- The array contains integers.
- The array can contain duplicate values.
- The array can be empty.
- The input array does not need to be sorted.

10.1.5 Important Edge Cases

Case	Example
Empty array	$[]$
One element	$[5]$
Already sorted	$[1, 2, 3, 4]$
Duplicate values	$[2, 2, 1]$
Reverse sorted	$[4, 3, 2, 1]$

10.2 2. Structure

A modified Merge Sort is used to count the inversions.

Normally, Merge Sort divides the array into smaller parts and then merges them back in sorted order.

While merging two sorted parts, we can also count inversions.

Suppose the left part and right part are already sorted.

If

$$left[i] \leq right[j],$$

we copy the left value.

If

$$left[i] > right[j],$$

then the current right value is smaller than the current left value and all the remaining values in the left part.

Therefore, the number of new inversions is

$$mid - i + 1.$$

This allows several inversions to be counted at once instead of checking every pair separately.

10.2.1 Selected Approach

Approach	Modified Merge Sort
Main Idea	Count inversions while merging sorted halves.
Why	It counts many inversions at once and gives $O(n \log n)$ time.

10.3 3. Pseudocode

```
1 Algorithm CountInversions(A)
2
3   if size of A is 0 or 1 then
4       return 0
```

```

5      end if
6
7      divide A into two halves
8
9      count the left inversions
10     count the right inversions
11     count the inversions while merging the two halves
12
13     return total count
14
15 End Algorithm
16
17
18 Algorithm Merge(A, left, middle, right)
19
20     compare elements from the two halves
21
22     if left element <= right element then
23         add left element
24     else
25         add right element
26         add number of remaining elements in left half to count
27     end if
28
29     continue until both halves are merged
30
31     return count
32
33 End Algorithm

```

10.4 4. Correctness

During the merge step, both the left and right parts are already sorted.

If the current left value is smaller than or equal to the current right value, there is no inversion between them, so the left value is added to the merged array.

If the current left value is greater than the current right value, the right value is smaller than all the remaining values in the left part.

If there are

$$mid - i + 1$$

elements remaining in the left part, all of them form an inversion with the current right value.

Therefore, adding

$$mid - i + 1$$

counts all the new inversions found during that merge.

The algorithm counts inversions inside the left half, inside the right half, and between the two halves.

Therefore, every inversion pair is counted exactly once.

10.5 5. Cost Model

Let n be the number of elements.

Operation	Cost
$O(\log n)$ levels	Divide the array
Merge at each level	$O(n)$ height
$O(n \log n)$	Total time
Extra space	$O(n)$ height

Merge Sort divides the array into smaller halves.

There are about

$$O(\log n)$$

levels of division.

At each level, all elements are processed during merging, so the work is

$$O(n).$$

The costs can be multiplied together:

$$T(n) = O(n) \cdot O(\log n).$$

Therefore, the total time is

$$T(n) = O(n \log n).$$

The temporary array used during merging requires

$$O(n)$$

extra space.

The inversion count is stored using a 64-bit integer.

10.6 6. Validation

10.6.1 Test Case 1: Provided Example

Input:

$$A = [2, 4, 1, 3, 5]$$

The inversion pairs are

$(2, 1), (4, 1), (4, 3).$

Therefore, the total number of inversions is

3.

Expected Output:

3

Actual Output:

3

The output matches the given example.

10.6.2 Test Case 2

Input:

$A = [4, 1, 3, 2]$

The inversion pairs are

$(4, 1), (4, 3), (4, 2), (3, 2).$

Therefore, the total number of inversions is

4.

Expected Output:

4

Actual Output:

4

The output is correct.

10.6.3 Test Case 3: Edge Case

Input:

$A = [5, 5, 5]$

There are no inversions because equal values do not satisfy

$A[i] > A[j].$

Expected Output:

0

Actual Output:

0

The algorithm correctly handles duplicate values.

```
De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem10.cpp -o Problem10
cc1plus.exe: fatal error: Problem10.cpp: No such file or directory
compilation terminated.

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ g++ Problem10.cpp -o Problem10

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$ ./Problem10
Test Case 1
Input: [2,4,1,3,5]
Output: 3

Test Case 2
Input: [4,1,3,2]
Output: 4

Test Case 3
Input: [5,5,5]
Output: 0

De11@DESKTOP-KG7GLAQ UCRT64 /c/Users/De11/Downloads/Assignment01_machado
$
```

Figure 23: Problem 10 execution screenshot.